Learn Python for Automation

Lesson 1: Introduction to Python and Setup

Introduction to Python and Setup - Study Notes

Key Concepts

1. **Python Installation** – How to download, install, and verify Python on Windows, macOS, and Linux.
2. **Integrated Development Environments (IDEs) & Text Editors** – Choosing between VS /Code, PyCharm, Jupyter, and lightweight editors.
3. **Running Python Code** – Executing scripts from the command line, IDE run buttons, and interactive shells.
4. **Basic Syntax** – `print()` statements, comments (`#`), and the importance of indentation.
5. **Environment Management** – Using `venv` or `conda` to create isolated project spaces.

Summary

Python is a versatile, high level programming language known for its readability and vast ecosystem. The first step in any Python journey is setting up a reliable development environment. This lesson walks you through installing Python from the official website, verifying the installation, and choosing an IDE or editor that fits your workflow. Once you have your environment ready, you'll learn how to write and run a simple "Hello, World!" script, understand how comments help document code, and see how the Python interpreter evaluates `print()` statements. These fundamentals lay the groundwork for deeper exploration into variables, data types, and control flow.

Important Points

- **Python 3.x** is the current standard; Python 2 is end of life.
- The installer adds Python to your system `PATH` (Windows) or updates shell profiles (macOS/Linux).
- Use `python --version` or `python3 --version` to confirm the correct interpreter is active.
- IDEs like VS /Code or PyCharm provide autocompletion, linting, and debugging; lightweight editors (Sublime, Atom) are fine for quick scripts.
- Create a project folder and a virtual environment (`python -m venv venv`) to keep dependencies isolated.
- Scripts are executed with `python script.py` (or `python3 script.py` on macOS/Linux).
- `print()` outputs to the console; use `#` for single line comments and `"""` or `''' ` for multi line docstrings.
- Indentation (4 spaces by convention) defines code blocks; mixing tabs and spaces can cause `IndentationError`.

Examples

1. Hello World Script

```python
# hello.py

print("Hello, World!")  # Prints a greeting to the console
```

Explanation:

- The `print()` function sends the string `"Hello, World!"` to standard output.
- The line starting with `#` is a comment; it is ignored by Python but useful for documentation.

2. Using a Virtual Environment

```bash
# Create a new project directory

mkdir my_python_project
cd my_python_project
# Initialize a virtual environment

python -m venv venv
# Activate it (Windows)

venv\Scripts\activate
# Activate it (macOS/Linux)

source venv/bin/activate
# Verify the Python interpreter

python --version
```

Explanation:

- `venv` creates an isolated copy of the Python interpreter and libraries.
- Activation prefixes the shell prompt with `(venv)` indicating that any packages installed will be local to this project.

Quick Review

1. What command would you run to check that Python is installed correctly?
2. Name two popular IDEs for Python development.
3. How do you write a single line comment in Python?
4. Why is it important to use a virtual environment?

5. What does the `print()` function do?

Further Reading

- **Python Basics** – data types, variables, and operators
- **Control Flow** – `if`, `for`, `while` statements
- **Functions** – defining and calling functions, arguments, return values
- **Modules & Packages** – importing standard library modules, creating your own modules
- **Error Handling** – `try`, `except`, `finally` blocks
- **Virtual Environments & Pip** – managing project dependencies
- **Version Control** – Git basics for Python projects

Happy coding! Ø=Þ€